

Kontrol ve Manevra Sistemi — Mimari Tasarım Kılavuzu

Hedef Kitle: Enerji mühendisleri, saha operasyon ekibi, proje yöneticileri

Amaç: Manevra kartlarının neden "sadece bir buton"dan ibaret olmadığını anlatmak.

İçindekiler

1. [Temel Kavramlar: Komut ve Manevra](#)
2. [Üç Katmanlı Mimari](#)
3. [Bir Manevranın Yolculuğu — FL-06 Örneği](#)
4. [Neden Bu Mimaride Israr Ediyoruz?](#)
5. [Low-Code Yol Haritası](#)
6. [Kartlar Neden Tek Başına Anlamsız?](#)
7. [Manevra Kataloğu](#)
8. [Yeni Manevra Ekleme Süreci](#)

1. Temel Kavramlar: Komut ve Manevra

Bu sistemde iki temel yapı taşı vardır. Bunlar sıklıkla karıştırılır — aralarındaki farkı netleştirmek gerekir.

Komut (Command)

Tek bir cihaza gönderilen tek bir işlemdir.

Örnek	Cihaz	Ne Yapar?
<code>charge</code>	BSC-1	Şarj moduna geç, güç setpoint'ini ayarla
<code>open</code>	CB-1	DC kesiciyi aç
<code>on</code>	HVAC-3	Klimayı çalıştır
<code>force_cool</code>	HVAC-5	Soğutmayı maksimuma zorla

Her komut **cihazın JSON konfigürasyon dosyasında tanımlıdır**. Bu dosyada şunlar yazar:

- Hangi Modbus register'ına yazılacak
- Hangi değer yazılacak
- Parametre gerekiyor mu (örn. `powerKw`)
- Yazdıktan sonra hangi register okunup doğrulanacak (`validate`)
- İşlem atomik mi — yani başarısız olursa geri alınsın mı (`atomic`)

Örnek: BSC-1'in `charge` komutu (`config-docker/bsc-1.json` içinden):

```

"charge": {
  "label": "Şarj",
  "telemetries": [
    { "name": "Command Request", "value": 2 },
    { "name": "Charge Power Setpoint", "value": "{{powerKw}}" }
  ],
  "params": {
    "powerKw": { "type": "number", "min": 0, "max": 3568, "default": 50 }
  },
  "atomic": true,
  "validate": {
    "reads": [{ "name": "Request Acknowledge", "expect": 2 }]
  }
}

```

Bu ne demek? "BSC-1'in **Command Request** register'ına 2 yaz, **Charge Power Setpoint** register'ına kullanıcının girdiği güç değerini yaz. Sonra **Request Acknowledge** register'ını oku, 2 döndü mü diye kontrol et. Başarısız olursa eski değerlere geri dön."

Örnek: CB-1'in **close komutu** (`config-docker/cb-1.json` içinden):

```

"close": {
  "label": "Kapat",
  "telemetries": [{ "name": "Close", "value": 1 }],
  "validate": {
    "reads": [{ "name": "Is Closed", "expect": true }]
  }
}

```

"CB-1'in **Close** coil'ine 1 yaz. Sonra **Is Closed** discrete input'unu oku, **true** döndü mü diye kontrol et."

Manevra (Maneuver)

Birden fazla cihaza, belirli bir sıra ve kuralla gönderilen komut zinciridir.

Örnek	Kapsadığı Komutlar	Sıra
FL-06 Şarj	CB-1 → close, CB-2 → close, BSC-1 → charge, BSC-2 → charge	Önce kesiciler, sonra BSC'ler
FL-03 Acil Durdurma	BSC'ler → stop, DC'ler → off, CB'ler → open	Sırayla, tek tek
FL-01 Sistem Başlatma	CB'ler → open, BSC'ler → charge, DC'ler → on	Hepsi aynı anda

Her manevra **bir saha operasyon akış diyagramına (FL-01 ~ FL-11) birebir karşılık gelir**. Diyagramdaki her kutu, manevradaki bir adımdır.

Özet

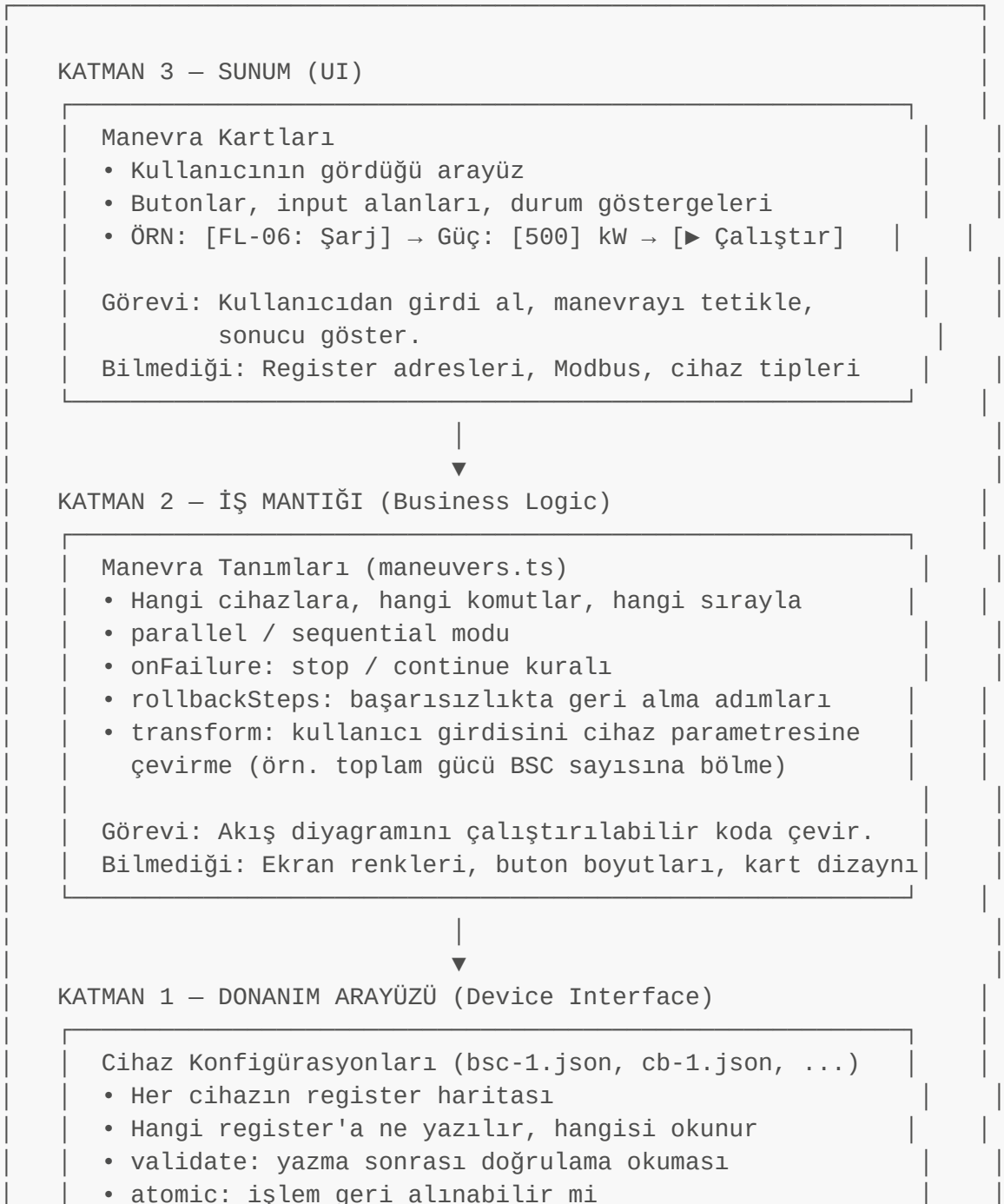
KOMUT: "BSC-1'e şarj sinyali gönder"
 MANEVRA: "Önce CB-1 ve CB-2'yi kapat, sonra BSC-1 ve BSC-2'yi 500kW ile şarja al"

Komutlar **donanım üreticisinin dilini** konuşur (Modbus register'ları, coil'ler).

Manevralar **saha operatörünün dilini** konuşur (sistemi başlat, şarja al, acil durdur).

2. Üç Katmanlı Mimari

Sistem yukarıdan aşağıya üç bağımsız katmandan oluşur. Her katmanın sorumluluğu farklıdır ve katmanlar birbirinin iç işleyişini bilmez.



- params: komutun aldığı parametreler

Görevi: Üretici dokümanındaki register'ları sisteme tanıtmak.

Bilmediği: Hangi manevranın bu komutu kullandığı, kullanıcı arayüzü

Katmanlar neden ayrı?

Bu üç katmanı birbirine karıştırmak, sistemi kısa vadede "hızlı" ama uzun vadede yönetilemez hale getirir. Bir benzetme:

Bir binanın elektrik tesisatı (Katman 1), sigorta panosu (Katman 2) ve duvardaki priz/anahtar (Katman 3) birbirinden bağımsız tasarlanır. "Bu odada 3 priz olsun" diye binanın ana elektrik hattını değiştirmesiniz. Aynı şekilde, "şarj ve deşarj tek kartta olsun" diye komut yapısını bozmazsınız.

3. Bir Manevranın Yolculuğu — FL-06 Örneği

En karmaşık manevralardan biri olan **FL-06: Şarj (Ön Kontrollü)** üzerinden tüm sistemi adım adım takip edelim.

3.1 Manevra Tanımı (Katman 2)

```
f106_charge: {
  name: "f106_charge",
  label: "FL-06: Şarj (Ön Kontrollü)",
  mode: "sequential",           // ← adımlar sırayla çalışsın
  onFailure: "stop",           // ← biri başarısızsa dur
  steps: [...cbClose(), ...bscCharge(500)],
}
```

Bu ne anlama geliyor?

1. CB-1 → close komutu
2. CB-2 → close komutu
3. BSC-1 → charge komutu (500 kW)
4. BSC-2 → charge komutu (500 kW)

1 ve 2 başarısız olursa → 3 ve 4 hiç çalışmaz (onFailure: "stop")

3.2 Kullanıcı Karttan Çalıştır'a Bastığında (Katman 3 → 2)

Kullanıcı **MANEUVER_CONTROLS** sayesinde kartta bir güç değeri girebilir:

```

MANEUVER_CONTROLS.fl_bsc_power = {
  inputs: [
    { name: "mode", label: "Mod", type: "select",
      options: [{ value: 0, label: "Şarj" }, { value: 1, label: "Deşarj" }]
    },
    { name: "powerKw", label: "Güç", unit: "kW",
      min: 0, max: 3568, default: 50 },
  ],
  transform: (values, steps) => {
    const command = values.mode === 0 ? "charge" : "discharge";
    return steps.map(() => ({ ...values, command }));
  },
}

```

transform fonksiyonu kritiktir: Kullanıcının girdiği `mode: 0`, `powerKw: 500` değerlerini, her bir BSC adımı için ayrı ayrı parametrelere dönüştürür. `steps.map(() => ...)` sayesinde "her cihaza aynı gücü ver" mantığı tek satırda ifade edilir.

3.3 API'ye Ulaşma (Katman 2 → Katman 1)

Manevra adımları `POST /api/commands/execute-multi` endpoint'ine şu JSON olarak gider:

```

{
  "commands": [
    { "deviceId": "CB-1", "command": "close" },
    { "deviceId": "CB-2", "command": "close" },
    { "deviceId": "BSC-1", "command": "charge", "params": { "powerKw": 500 } },
    { "deviceId": "BSC-2", "command": "charge", "params": { "powerKw": 500 } }
  ],
  "mode": "sequential",
  "onFailure": "stop"
}

```

3.4 Web-Service İşleme (Katman 1)

Web-service her adım için:

1. Cihazın JSON dosyasını açar (`config-docker/cb-1.json`)
2. `commands.close` tanımını bulur
3. `telemetries` listesini okur: `[{ name: "Close", value: 1 }]`
4. `validate` varsa doğrulama adımlarını ekler: `[{ name: "Is Closed", expect: true }]`
5. Bir **BullMQ işi (job)** oluşturur ve kuyruğa atar
6. İşin tamamlanmasını bekler (timeout süresi kadar)
7. Sonucu döner: `{ deviceId: "CB-1", success: true }`

3.5 Device-Service ve Modbus (Katman 1 — Donanım)

Device-service kuyruktan **COMMAND_DEVICE** işini alır:

1. **Atomic kontrolü:** Eğer **atomic: true** ise, ilgili register'ların mevcut değerlerini okur ve yedekler
2. **Yazma:** Modbus protokolü üzerinden ilgili register'lara değerleri yazar
3. **Doğrulama (validate):** Yazdıktan sonra belirtilen register'ları tekrar okur, beklenen değerle karşılaştırır
4. **Rollback:** Doğrulama başarısız olursa, yedeklenen eski değerleri geri yazar

3.6 Tüm Akışın Özeti

```

Kullanıcı kartta [▶ Çalıştır] tuşuna basar
↓
Manevra tanımı: sequential mod, stop on failure
↓
Web-service: her adım için cihaz JSON'ını oku, job oluştur
↓
BullMQ kuyruğu: COMMAND_DEVICE işleri
↓
Device-service: Modbus yaz → doğrulama oku → sonuç bildir
↓
Kullanıcıya sonuç: ✅ Başarılı / ❌ Başarısız – hangi cihaz, hangi adım

```

Bu zincirin hiçbir halkası "sadece bir buton" değildir. Her halka, bir öncekinden bağımsız, kendi sorumluluğu olan bir katmandır.

4. Neden Bu Mimaride Israr Ediyoruz?

4.1 Sürdürülebilirlik

Bir benzetmeyle başlayalım:

Bir fabrikanın üretim bandını düşünün. Banttaki her makinenin kendi kullanım kılavuzu vardır. Operatörler bu makineleri tek tek değil, "reçete" adı verilen iş akışlarıyla yönetir. "Ürün A Reçetesi": Makine 1 → 200°C, Makine 3 → 50 bar, Makine 5 → 30 sn.

Bu sistemde **cihaz JSON'ları = makine kullanım kılavuzu**, **manevralar = reçeteler**, **kartlar = operatör panelidir**.

Bu mimari sayesinde:

Değişiklik	Nereye Dokunulur?	Kaç Dosya?
Yeni bir BSC-3 cihazı eklendi	BSC_IDS dizisine " BSC-3 " ekle	1 satır
BSC'nin register adresi değişti	bsc-1.json ve bsc-2.json 'da ilgili register	2 dosya

Değişiklik	Nereye Dokunulur?	Kaç Dosya?
FL-06'nın sıralaması değişti	<code>maneuvers.ts</code> 'te <code>mode</code> veya <code>steps</code>	1 dosya
Kartın rengi mavi olsun	UI bileşeninin CSS'i	1 dosya
Tüm BSC'ler için yeni bir komut eklendi	<code>bsc-1.json</code> ve <code>bsc-2.json</code>	2 dosya
5 yeni manevra eklendi	<code>maneuvers.ts</code> 'e 5 yeni giriş	1 dosya

Değişiklik her zaman **tek bir katmanda, tek bir yerde** kalır. Diğer katmanlara sıçramaz.

4.2 Güvenlik Zinciri

Her komutun arkasında üç güvenlik mekanizması vardır:

Mekanizma	Ne Yapar?	Örnek
validate	Yazdıktan sonra okuyup doğrular	CB'ye "kapat" yazdın → <code>Is Closed</code> gerçekten <code>true</code> mu?
atomic	Başarısız olursa eski değerlere döner	BSC'ye charge yazdın ama acknowledge gelmedi → eski register değerlerini geri yaz
onFailure: stop	Zincirdeki bir hata tüm manevrayı durdurur	CB-1 kapanmadıysa BSC'leri şarja alma

Bu mekanizmalar **cihaz JSON'larında tanımlıdır**, manevra kodunda değil. Yani yeni bir manevra yazarken güvenlik kurallarını tekrar tekrar yazmak zorunda kalmazsınız — her komut kendi güvenliğini içinde taşır.

4.3 Akış Diyagramlarıyla Birebir Eşleme

Sahadaki her operasyonun bir **akış diyagramı (FL-01 ~ FL-11)** vardır. Bu diyagramlar `.drawio` formatında, enerji mühendisleri tarafından çizilmiştir.

Mimarinin temel kuralı: **Bir akış diyagramı = Bir manevra tanımı = Bir manevra kartı.**

```
.drawio diyagramı → maneuvers.ts girişi → UI'da otomatik kart
(mühendis)          (geliştirici)      (operatör)
```

Bu zinciri kırmak — örneğin "FL-05 ve FL-06'yı tek kartta birleştirelim" demek — diyagram ile kod arasındaki bağı koparmak, dolayısıyla sistemin izlenebilirliğini yok etmek demektir.

4.4 Yeniden Kullanılabilirlik

`charge` komutu BSC JSON'unda **bir kez** tanımlanır. Bu komutu kullanan manevralar:

- **FL-01** (Sistem Başlatma) — `bscCharge(0)` — sıfır güçle başlat
- **FL-04** (Kalibrasyon Şarj) — `bscCharge(500)` — 500 kW ile
- **FL-06** (Ön Kontrollü Şarj) — `bscCharge(500)` — 500 kW ile
- **fl_bsc_power** (Manuel Güç Kontrolü) — kullanıcının girdiği değerle

Register adresi değişse, sadece JSON güncellenir. 4 manevraya da tek tek dokunulmaz.

5. Low-Code Yol Haritası

Bu mimariyi kurmamızın bir diğer sebebi: **bugün geliştirici gerektiren işleri, yarın enerji mühendisinin kendi başına yapabilmesi.**

Bugün (Mevcut Durum)

Yeni manevra eklemek için:

1. maneuvers.ts'e ManeuverConfig girişi yaz (geliştirici)
2. MANEUVER_CONTROLS'a input tanımı ekle (geliştirici)
3. Kart otomatik render edilir (otomatik)

Yarın (Planlanan — Low-Code)

Yeni manevra eklemek için:

1. Web arayüzünden form doldur:
 - Manevra adı: FL-12
 - Cihazlar: BSC-1, BSC-2, HVAC-1
 - Komutlar: charge (seç), force_cool (seç)
 - Mod: sequential
 - Input alanı: Güç (kW), min: 0, max: 3568
2. Kaydet → maneuvers.ts otomatik güncellenir
3. Kart anında görünür

Bunu mümkün kılan şey, **tüm sistemin veri odaklı (data-driven) olmasıdır**. Kartlar manuel kodlanmaz; bir **ManeuverConfig** objesinden otomatik üretilir. Bugün bu objeyi biz TypeScript dosyasında yazıyoruz. Yarın aynı objeyi bir web formu üretecek.

Low-Code'un temel taşları (bugünden hazır):

Yapı Taşı	Bugün Nerede?	Yarın Nerede Olacak?
Cihaz komut tanımları	JSON dosyaları	Aynı JSON — web'den düzenlenebilir
Manevra tanımları	maneuvers.ts (TypeScript)	Veritabanı — web'den CRUD
Input alanları	MANEUVER_CONTROLS (TypeScript)	Veritabanı — form builder
Kart render'ı	ManeuverCard bileşeni (React)	Aynı bileşen — veri nereden gelirse gelsin çalışır
Validasyon kuralları	Cihaz JSON'ı içinde	Aynı — zaten veri odaklı

Mimari şu an low-code'a hazır. Sadece veri kaynağını değiştirmek kaldı.

6. Kartlar Neden Tek Başına Anlamsız?

Sıkça karşılaştığımız istekleri ve neden mevcut mimaride karşılıklarının olmadığını açıklayalım.

İstek 1: "Şarj vedeşarj tek kartta olsun"

Neden olmaz: Şarj vedeşarj **farklı akış diyagramlarıdır**. FL-06'da önce CB'ler kapatılır, sonra BSC'ler şarja alınır. FL-06 Deşarj'da aynı sıra izlenir ama farklı komut çalışır. İkisi aynı kartta birleşirse:

- Hata durumunda hangi akışın başarısız olduğu belirsizleşir
- **rollbackSteps** (geri alma) adımları farklıdır — şarjı durdurmak iledeşarjı durdurmak farklı register'lar gerektirir
- Log kayıtları karışır — "FL-06 çalıştı" kaydı şarj mıdeşarj mı belirsiz olur
- Low-code'a geçince form mantığı karmaşıklaşır

Doğru yaklaşım: **fl_bsc_power** manevrası zaten şarj/deşarj seçimi sunar — ama bu **manuel kontrol** içindir, FL-06'nın kontrollü akışının yerine geçmez. Farklı senaryolar, farklı kartlar.

İstek 2: "DC Kesici Kapat ve Kontaktör Kapat tek kartta olsun"

Neden olmaz: CB (DC kesici) ve BSC (kontaktör) **farklı cihaz tipleridir, farklı komut setlerine sahiptir, farklı güvenlik zincirleri vardır**.

- CB **close**: Coil register → **Is Closed** okumasıyla doğrulanır
- BSC **contactor_close**: Holding Register → **Request Acknowledge** ile doğrulanır

Bu iki işlemi tek kartta birleştirmek, iki ayrı akış diyagramını tek bir arayüzde eritmek demektir. Bugün "2'si bir arada" yaparsak, yarın "3'ü bir arada", sonra "hepsi bir arada" gelir. Her seferinde kartın iç mantığı büyür, test edilemez hale gelir.

Doğru yaklaşım: **fl_dc_breaker_close** ve **fl_contactor_close** ayrı kartlardır. İkisi de **parallel** modda çalışır, kendi cihaz grubuna komut gönderir. Operatör ihtiyacı olanı seçer.

İstek 3: "BSC-1 ve BSC-2 için ayrı ayrı kart olsun"

Neden olmaz: Manevralar **cihaz grubu bazında** tasarlanır. BSC-1 ve BSC-2 **her zaman birlikte** hareket eder — aynı şarj/deşarj gücünü paylaşırlar. Ayrı kartlar:

- Paralel çalışan cihazların senkronizasyonunu bozar
- **transform** mantığını gereksiz yere karmaşıklaştırır
- Operatör hatası riskini artırır ("BSC-1'i şarja aldım, BSC-2'yi unuttum")

Doğru yaklaşım: BSC grubu (**BSC_IDS = ["BSC-1", "BSC-2"]**) tek bir manevrada yönetilir. **steps.map(() => ...)** sayesinde her cihaza aynı parametreler otomatik dağıtılır.

Genel Kural

Bir kart, bir akış diyagramına karşılık gelir.

Kartlar keyfi olarak bölünmez, birleştirilmez.

Kartın içeriğini UI tasarımı değil, saha operasyon mantığı belirler.

7. Manevra Kataloğu

Sistemde şu anda **20 manevra** tanımlıdır. Her biri bir saha operasyon senaryosuna karşılık gelir.

Sistem Seviyesi Manevralar

Kod	Etiket	Amaç	Cihazlar	Mod
FL-01	Sistem Başlatma	Tüm sistemi sıfırdan ayağa kaldır	CB(2) + BSC(2) + DC(2)	parallel
FL-02	AUX Kaybı	Yardımcı besleme kaybında kontrollü durdurma	BSC(2) + DC(2) + CB(2)	sequential
FL-03	Acil Durdurma	Emergency stop butonu ile tam kapatma	BSC(2) + DC(2) + CB(2)	sequential
FL-10	Bakım Modu	Planlı bakım için güvenli kapatma	BSC(2) → CB(2)	sequential

BSC (Batarya) Manevraları

Kod	Etiket	Amaç	Cihazlar	Mod
FL-04	Kalibrasyon Şarj	0.25C ile kontrollü kalibrasyon şarjı	BSC(2)	parallel
FL-04	Kalibrasyon Deşarj	0.25C ile kontrollü kalibrasyon deşarjı	BSC(2)	parallel
FL-06	Şarj (Ön Kontrollü)	Kesicileri kapatıp şarja al	CB(2) → BSC(2)	sequential
FL-06	Deşarj (Ön Kontrollü)	Kesicileri kapatıp deşarja al	CB(2) → BSC(2)	sequential
fl_bsc_power	BSC Güç Kontrolü	Manuel şarj/deşarj + güç ayarı	BSC(2)	parallel
fl_idle	Durdur	Tüm BSC'leri durdur	BSC(2)	parallel

TMS (Termal Yönetim) Manevraları

Kod	Etiket	Amaç	Cihazlar	Mod
FL-05	Soğutmayı Zorla	8 HVAC'ı soğutma moduna al	HVAC(8)	parallel
FL-05	Isıtmayı Zorla	8 HVAC'ı ısıtma moduna al	HVAC(8)	parallel
FL-05	Termal Koruma Durdur	Sıcaklık limit aşımında BSC'leri durdur	BSC(2)	parallel

Güvenlik Manevraları

Kod	Etiket	Amaç	Cihazlar	Mod
FL-07	Kapı Açık	Panel kapısı açıldığında otomatik durdurma	BSC(2)	parallel
FL-08	DC Arıza	DC tarafı arızasında koruma kapatma	BSC(2) + CB(2)	parallel
FL-09	İletişim Kaybı	Haberleşme koptuğunda güvenli durdurma	BSC(2)	parallel
FL-11	Toprak Direnci Hatası	Topraklama direnci anormal — kapatma	BSC(2) → CB(2)	sequential

Cihaz Seviyesi Manevralar

Kod	Etiket	Amaç	Cihazlar	Mod
fl_dc_breaker_close	DC Kesici Kapat	CB-1 ve CB-2'yi kapat	CB(2)	parallel
fl_contactor_close	Kontaktör Kapat	BSC kontaktörlerini kapat	BSC(2)	parallel

8. Yeni Manevra Ekleme Süreci

Bir manevra ekleme süreci şu adımları izler:

Adım 1: Akış Diyagramı

Önce .drawio'da FL-XX diyagramı çizilir. Hangi cihazlar, hangi sırayla, hangi koşullarda çalışacak — netleşir.

Adım 2: Cihaz Komutu Kontrolü

Diyagramdaki her adım için ilgili cihazın JSON'ında komut var mı kontrol edilir. Yoksa önce komut eklenir:

```
// Örn: BSC'ye yeni bir "precharge" komutu


```

"precharge": {
 "label": "Ön Şarj",
 "telemetries": [
 { "name": "Command Request", "value": 22 },
 { "name": "Precharge Current", "value": "{{currentA}}" }
],
 "params": {
 "currentA": { "type": "number", "min": 0, "max": 100, "default": 10 }
 },
 "validate": {
 "reads": [{ "name": "Request Acknowledge", "expect": 2 }]
 }
}

```


```

Adım 3: Manevra Tanımı

`maneuvers.ts`'e `ManeuverConfig` girişı eklenir:

```
fl12_precharge: {
  name: "fl12_precharge",
  label: "FL-12: Ön Şarj",
  mode: "sequential",
  onFailure: "stop",
  steps: [...cbClose(), ...bscPrecharge(10)],
  rollbackSteps: bscStop(),
}
```

Adım 4: Input Alanları (varsa)

Kullanıcıdan parametre alınacaksa `MANEUVER_CONTROLS`'a giriş eklenir:

```
fl12_precharge: {
  inputs: [
    { name: "currentA", label: "Akım", unit: "A",
      min: 0, max: 100, default: 10 },
  ],
}
```

Adım 5: Test

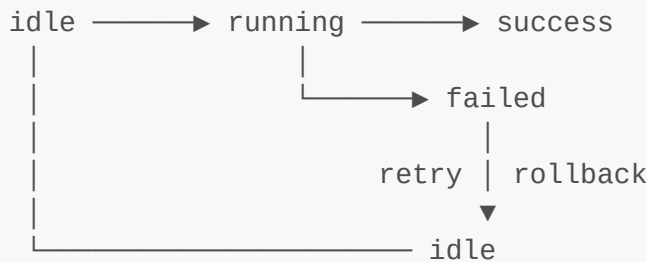
Manevra otomatik olarak UI'da kart olarak belirir. Test edilir.

Bu sürecin kritik noktası

Adım 4'ten sonra yeni UI kodu yazılmaz. Kart, `ManeuverCard` bileşeni tarafından `ManeuverConfig` verisinden otomatik üretilir. Bu, low-code geçişinin temelidir.

Ek: Kart Durum Makinesi

Her manevra kartı şu durumlardan geçer. Bu durumlar UI'da farklı buton ve göstergelerle temsil edilir:



Durum	Gösterge	Butonlar
idle	Mavi — hazır	► Çalıştır (Şimdi / Zamanlı)
running	Sarı — çalışıyor	(butonlar pasif)
success	Yeşil — başarılı	► Tekrar Çalıştır
failed	Kırmızı — hata	Tekrar Dene / Geri Al

Sonuç

Bu sistemde **kart, tasarımın son halkasıdır — ilk halkası değil**. Kartı değiştirmek için önce altındaki katmanları anlamak gerekir. "Charge/Discharge tek kartta olsun" demek, aslında "FL-04 ve FL-06 akış diyagramlarını birleştirelim, güvenlik zincirini yeniden tasarlayalım, rollback mantığını değiştirelim" demektir. Bu mümkündür — ama bir UI değişikliği değil, bir **sistem tasarımı** değişikliğidir.

Mimari, bugün 20 manevra için ne kadar sağlamsa, yarın 200 manevra için de o kadar sağlam kalacak şekilde tasarlanmıştır. Her yeni cihaz, her yeni komut, her yeni manevra — sisteme bir tuğla daha ekler, mevcut tuğlaları yerinden oynatmaz.